Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: ‹Team name›
b. Team leader name: ‹Team leader name›
c. Problem Statement: Retail & Commerce. A shopper can't tell which nearby shop really has the exact item in stock, and small shops can't show it, so trips are wasted and sales lost.

NeedProof

Know it's in stock before you go.

Real nearby shops prove it with one photo; Gemini checks the proof.

Brief about the idea

Know it's in stock before you go

The problem

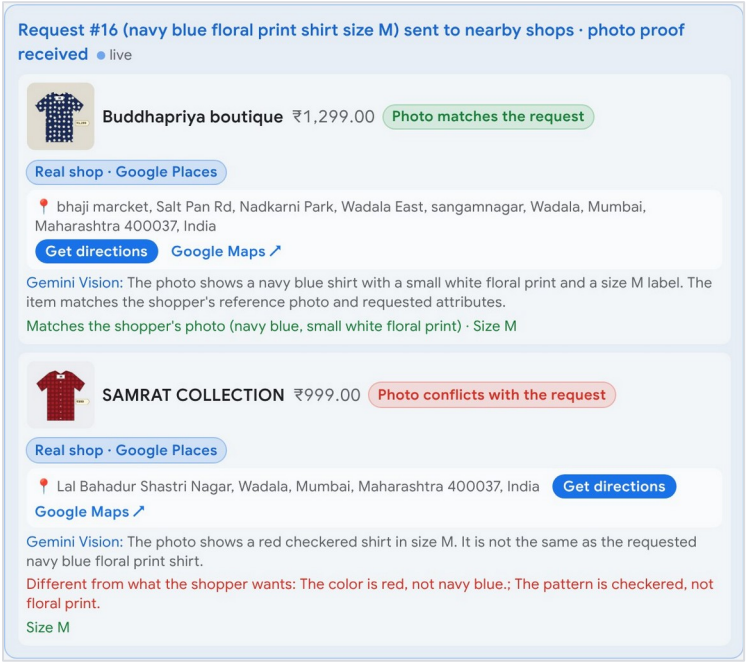
A shopper needs a specific item today: a 65W charger, a shirt in this design, a mixer grinder. Online listings and maps say what a shop sells, not what is on its shelf now. So people phone around or travel and find it missing, and the shop never learns it lost a sale.

What NeedProof does

The shopper asks in any language, by text, voice or photo. Gemini works out the item and the kind of shop that sells it, and the request goes to real nearby shops from Google Maps. A shop answers with one photo, without installing anything. Gemini Vision checks the photo against the request, and the shopper goes only where the item is proven, with directions.

What the business gets

Every request becomes demand data: what people asked for, what was missed, which shops answered. Shopkeepers see their own demand, missed sales and stock cover; managers see demand by category with BigQuery ML forecasts.



Live app: one shop's photo matches the request, another's conflicts. Both are real shops from Google Places.

Opportunities, differentiation and USP

Opportunity

- Most neighbourhood shops have no online catalogue, so shoppers phone around or walk in blind
- Listings and maps say a shop sells something, not that the exact item is there today
- An unanswered ask is a lost sale, and demand data the shop never sees

How it is different

- Marketplaces show listings (claims); maps show shops, not stock; AI chat answers from the web
- NeedProof asks the actual shop, now, for one photo, and Gemini Vision checks design, size, specs and brand
- Works for shops with no app, no catalogue and no account: a one-tap WhatsApp link

How it solves the problem

- The shopper travels only to a shop that proved it has the item
- The shop gets a ready-to-buy customer and sees demand it was missing
- The business sees demand by category, missed sales and forecasts

USP Proof, not promises. Before you travel, a real nearby shop shows the exact item with one photo, and Gemini checks it matches what you asked for.

Features offered by the solution

1 Ask any way

Text, voice or a photo, in any language. Gemini extracts item, size, budget and place.

Gemini

2 Search by photo

"Find this design": the shopper's photo travels with the request to shops.

Gemini Vision

3 AI routing

Gemini picks the kind of shop that sells the item (up to two); only those inboxes get it.

Gemini

4 Real nearby shops

Open shops of that kind from Google Places, with rating, Maps link and directions.

Places API

5 One-photo reply

From the inbox or a one-tap WhatsApp link, no app or account. AI drafts from stock; a person approves.

Vertex AI Search

6 Photo proof check

Model reads the photo; code compares design, colour, size, specs and brand: match or conflict.

Gemini Vision

7 Shopkeeper insights

Own demand and missed sales; forecast, customer groups and stock cover from its own sales file.

ML on shop data

8 Business intelligence

Requests routed by category, city demand, BigQuery ML forecast and a BigQuery AI brief.

BigQuery ML · AI

Also built: multi-item basket optimiser (OR-Tools CP-SAT) · proof planner (EVSI) · remembers shopper preferences with "forget me" · four roles checked by the server · Trust report with live tests

Process flow and use cases

SHOPPER
1 Ask
"Navy floral shirt like this, size M, under ₹1500, near Wadala" by text, voice or photo, in any language

→

NEEDPROOF
2 Understand + route
Reads the photo and request; picks the kind of shop that sells it (clothing)
Gemini

→

NEEDPROOF
3 Find real shops
Open shops of that kind near the shopper; the request reaches only their inboxes
Places API · EVSI · CP-SAT

→

REAL NEARBY SHOP
4 Reply with a photo
From the inbox or a one-tap WhatsApp link, no app or account. AI drafts from stock; a person approves
Vertex AI Search

→

NEEDPROOF
5 Check the proof
Model reads the photo; code compares design, colour, size, specs, brand → match or conflict
Gemini Vision

→

SHOPPER
6 Go with confidence
Sees which shop proved it, its photo and price; gets directions
Google Maps

7 Every request teaches the business
Shopkeeper: demand for its category, missed sales, stock cover, forecast from its own sales. Business manager: routing by category, city demand, BigQuery ML forecast and BigQuery AI brief.

Use cases by role (picked from the role dropdown; the server checks role and shop on every private action)

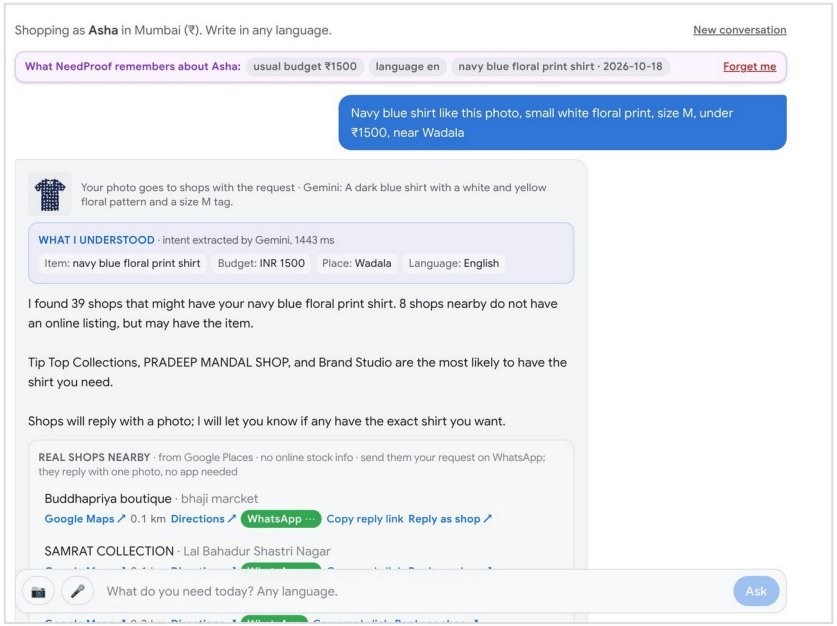
Shopper
Asks, sends a photo, compares proof, gets directions

Shopkeeper
Replies with a photo, uploads stock and sales, sees own demand

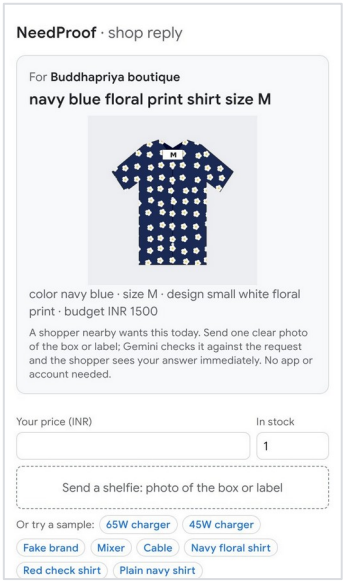
Business manager
Demand by category, forecasts, BigQuery AI brief

Admin / jury
Every screen; Trust report with live tests

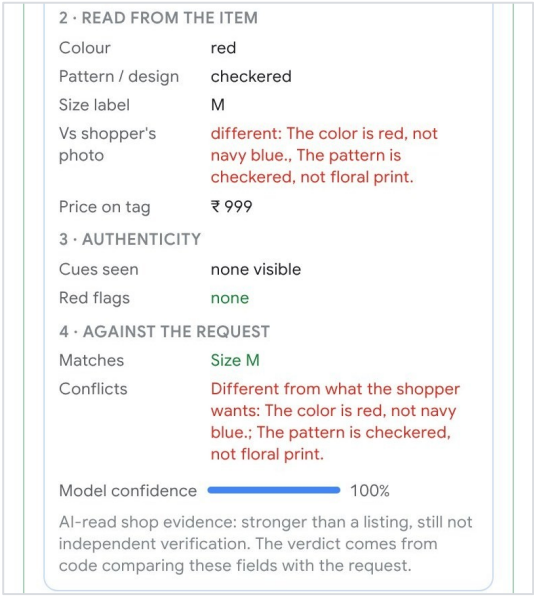
UI of the main journey (screens from the live app)



1 Shopper asks with a photo. Gemini shows what it understood; real shops of that kind are listed

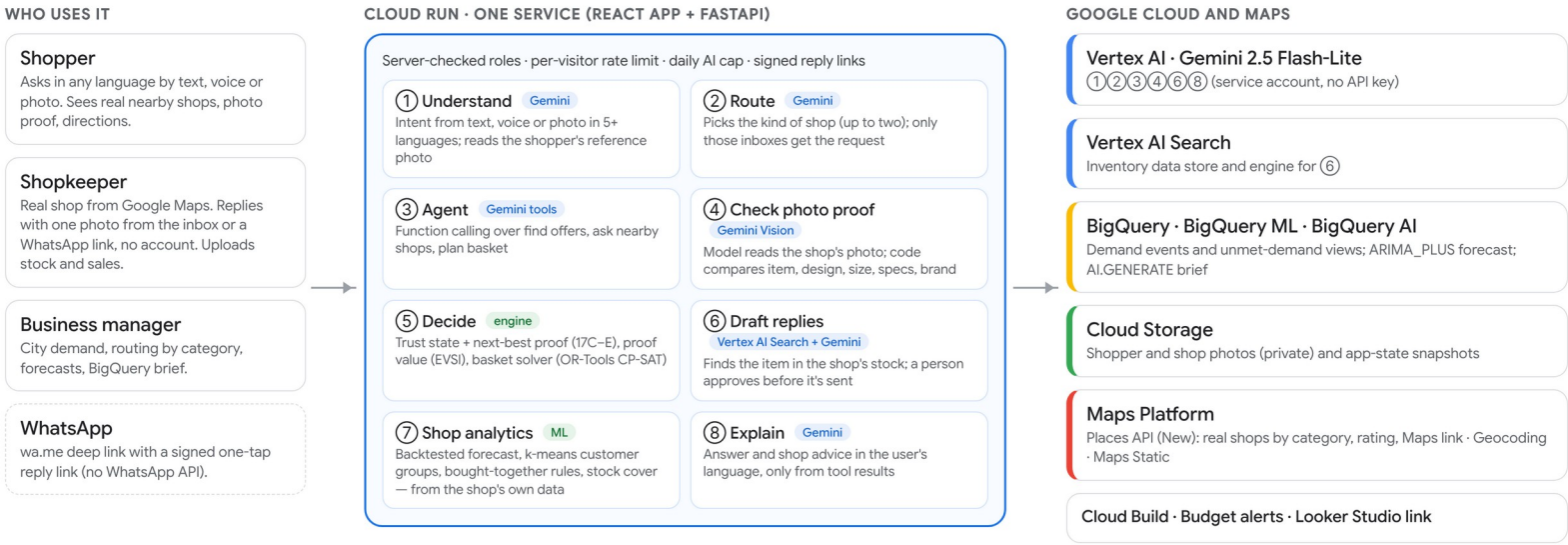


2 The shop opens the link: request and shopper's photo; replies with one photo



3 Gemini Vision reads the reply. A red check shirt is flagged as a conflict, with reasons

Architecture



One Cloud Run service (max 1 instance) serves the React app and the FastAPI API. Gemini proposes; code and the decision engine decide. Not used: Firebase, Firestore, AlloyDB/Cloud SQL.

Technologies used

Google Cloud and Maps

- **Gemini 2.5 Flash-Lite on Vertex AI**
  Intent, routing, agent tool calls, photo reading, drafts, advice
- **Vertex AI Search**
  Finds the requested item in a shop's uploaded stock
- **BigQuery · BigQuery ML · BigQuery AI**
  Demand views, ARIMA_PLUS forecast, AI.GENERATE brief
- **Cloud Run + Cloud Build**
  One container: React app and FastAPI API, built from source
- **Cloud Storage**
  Shopper and shop photos (private) and app-state snapshots
- **Maps Platform**
  Places API (New) real shops · Geocoding · Maps Static
- **Looker Studio**
  Report on the BigQuery views (link prepared)

Engineering

Backend Python, FastAPI, google-genai SDK, Pydantic schemas for every model output

Frontend React 19, Vite, Tailwind; Google Sans; role-based screens

Decision engine Trust states and next-best proof (17C–E), EVSI proof planner, OR-Tools CP-SAT basket solver

Analytics Backtested forecasting (4 baselines), k-means customer groups, association rules, stock cover

Data SQLite on Cloud Run, snapshotted to Cloud Storage; BigQuery for analytics

Quality pytest (546 tests), Playwright end-to-end runs on the live site, GitHub Actions CI

How NeedProof covers the theme's suggested directions and technologies

Suggested direction		Where in NeedProof
Conversational commerce	●	Gemini agent chat in any language, voice and photo; shops reply by link
Product discovery	●	Photo search, real shops by category, inventory search, photo proof
Hyper-personalization	◐	Remembers budget, device, past needs, language (with forget me)
Demand forecasting	●	Backtested per-shop forecast; BigQuery ML ARIMA_PLUS city demand
Basket analysis	●	Bought-together rules (confidence, lift); multi-item basket solver
Customer segmentation	●	k-means groups on recency, orders, spend from a shop's sales
Inventory intelligence	●	Stock upload, AI reply drafts, stock cover, missed sales
Fraud (problem statement)	◐	Flags fake brands and wrong items in shop photos

● built and live ◐ partly ○ not yet

Suggested technology		How it is used
Gemini on Vertex AI	●	Seven uses: photo, intent, routing, agent, proof, drafts, answers
Vertex AI / Agent Search	●	Inventory data store and engine
BigQuery	●	Demand events, unmet-demand views
BigQuery AI	●	AI.GENERATE brief + BigQuery ML forecast
Cloud Run	●	App and APIs
Cloud Storage	●	Photos and state snapshots
AlloyDB / Cloud SQL	○	Not yet: SQLite + snapshots (next step)
Looker	◐	Looker Studio link on BigQuery views

Estimated implementation cost

Cost per shopper request (list prices, Oct 2026)

Gemini 2.5 Flash-Lite	≈ US$0.0013	~5k input + 0.7k output tokens for the ask (measured), plus one photo check; $0.10 / $0.40 per 1M tokens
Places Nearby Search	US$0.035	Per uncached area + category search (Enterprise fields: phone, rating); shops are stored and reused
Geocoding	US$0.005	Only when the shopper types a place
Vertex AI Search	US$0.0015	Per stock search when a shop drafts a reply; 10,000 free queries a month
Cloud Run	≈ US$13 / month	One always-on instance at idle rates, plus active time per request
BigQuery, Cloud Storage	≈ US$0	Within free tiers at pilot volume

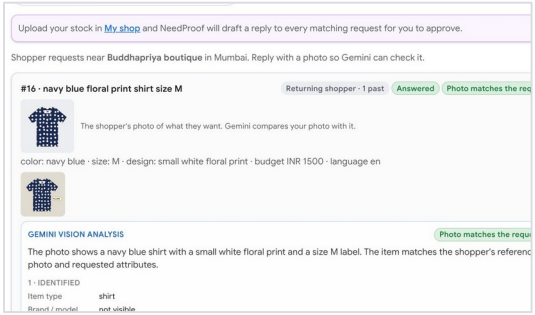
Pilot example: one city area, 10,000 requests a month

Gemini	≈ US$13
Places (30% uncached, 1,000 free)	≈ US$70
Cloud Run (always on)	≈ US$15–20
Search, BigQuery, Storage	≈ US$0–5
Total per month	**≈ US$100–110**

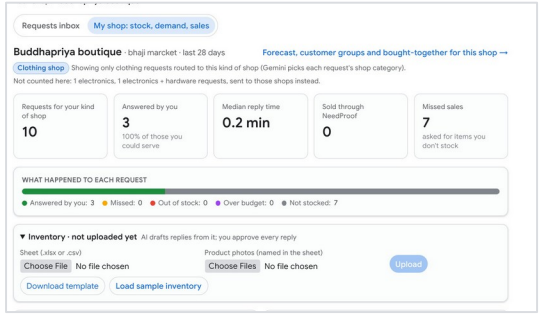
≈ US$0.01 per request. Maps search is most of it, and falls further when shops are refreshed per neighbourhood rather than per request.

Estimates from published list prices (Gemini, Places, Vertex AI Search, Cloud Run) and token counts measured on the live app; not a quote. Hackathon spend is capped by a ₹900 budget alert.

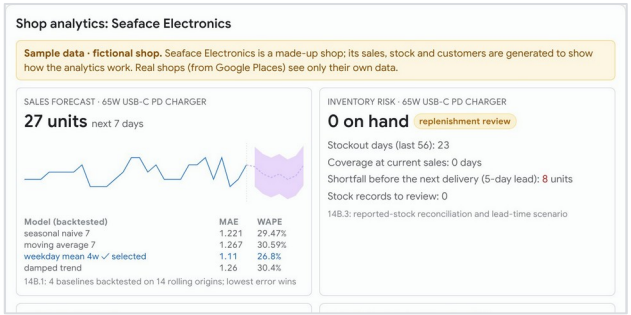
Snapshots of the prototype



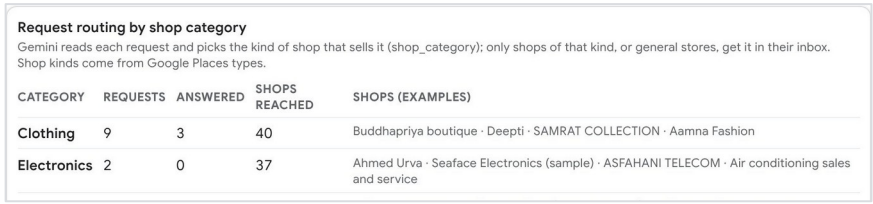
Shopkeeper inbox: the shopper's photo, the shop's photo and Gemini's check



My shop: requests for its kind of shop, answered, missed sales



Shop analytics: backtested sales forecast and stock risk (sample shop, labelled as such)



Business: requests routed by shop category

Prototype performance and benchmarking

19/19

live model tests pass on Vertex AI
10 requests in 5 languages incl. a prompt injection, 5 shop photos, 4 drafts (also 19/19 on AI Studio)

2.7 s · 3.4 s

median model time on Vertex AI
understanding a request · checking a shop photo (2.5 Flash-Lite)

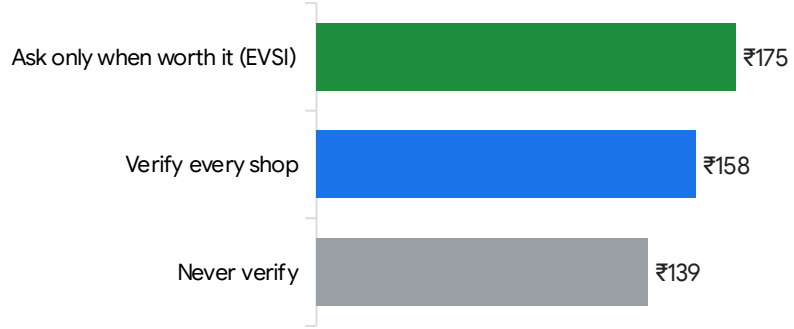
7–9 s

photo search, end to end on Cloud Run
photo read, intent, routing, Places search, agent answer (our runs; Vertex can be slower, with timeout and retry)

546

automated backend tests pass
plus lint, build and Playwright runs against the live site

When to ask a shop for proof (500 seeded trials)



Wrong picks: 4% with EVSI, 0% verifying all (2 checks each), 20% never verifying. EVSI averaged 0.93 checks.

Multi-item baskets (30 seeded scenarios)

OR-Tools found all 14 feasible baskets. Cheapest-item-first missed 3 and cost ₹4 more; a single shop missed 5 and cost ₹43 more.

Benchmarks are seeded simulations of the decision policy, not field results. Model tests use hand-written cases and synthetic label photos. All figures are reproducible from the repo and shown live in the Trust report.

Future development

Pilot

- 20–30 real shops in one Mumbai neighbourhood
- Measure reply rate, time to proof, trips saved, visits that became sales
- Shop trust score from proof history; repeated conflicts flagged

Product

- Shops reply inside WhatsApp (WhatsApp Business API)
- Notify a shopper when a shop restocks what they asked for
- Recommendations from a shopper's past needs and budgets

Platform

- Cloud SQL or AlloyDB for operational data; more Cloud Run instances
- Firebase Authentication for real shopper and shop sign-in
- Looker dashboards on the BigQuery views; Gemini check that filters mislabelled map listings

Business model Free for shoppers. Shops pay per ready-to-buy customer, or a monthly plan for demand and stock insights. Malls and brands buy category demand by neighbourhood.

Known limits today sample shops (labelled) carry fictional listings; tests are small; one Cloud Run instance.

Links

GitHub public repository	https://github.com/shrinivashunnur08/NeedProof-AI
Demo video (3–4 minutes)	‹add the video link›
Live prototype (Cloud Run)	https://needproof-822406860102.asia-southeast1.run.app

Try it: pick a role from "Signed in as". As a shopper in Mumbai, tap "Search by photo"; then "Reply as shop" on a real nearby shop to answer as that shopkeeper.

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

